

FINAL CASE STUDY

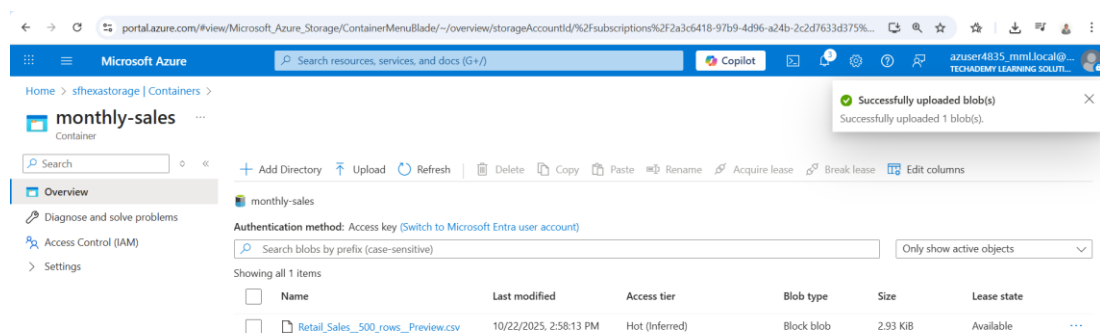
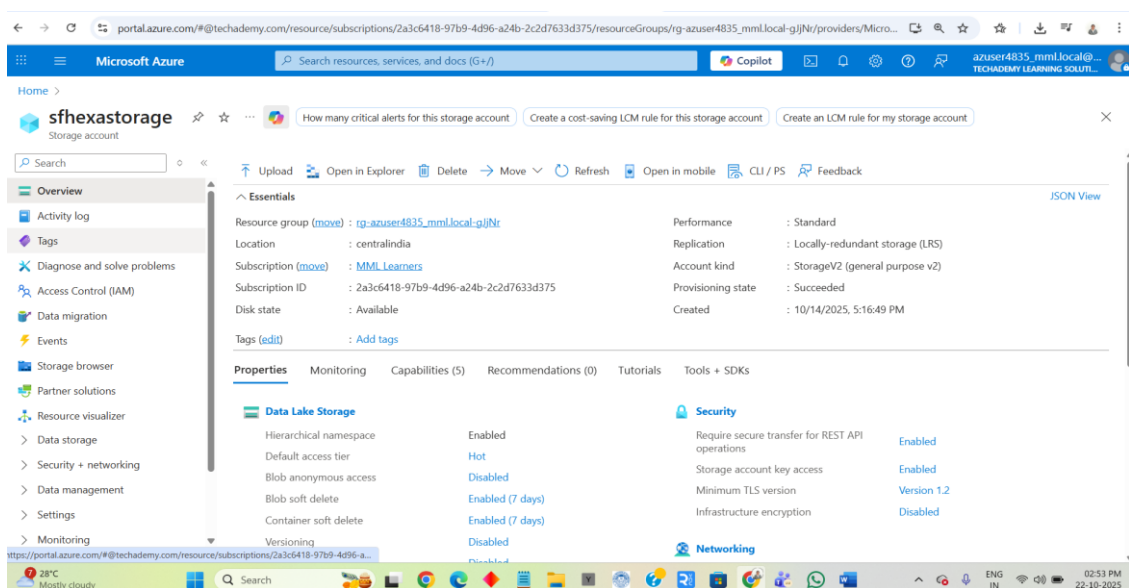
Case Study: Azure → Snowflake with Snowpark, then Power BI

Scenario

You're the data engineer at ItTechGenie Retail. Sales teams drop monthly CSVs into an Azure Storage container. You must:

1. upload the CSV to Azure,
2. ingest it into Snowflake using Snowpark,
3. model it into proper database/schema/table, and
4. build a quick Power BI report for business users.

Step 1: Create a storage account in azure portal and upload the data set into Azure Blob Storage container named monthly-sales



Step 2: Collect Required Snowflake Details

From your Snowflake account, note down the following:

Parameter	Example	Description
ACCOUNT	sqishot-fa68768	Snowflake account identifier (exclude .snowflakecomputing.com)
USER	poojashree	Snowflake login user
PASSWORD	Poojashree@307	Snowflake login password
WAREHOUSE	COMPUTE_WH	Virtual warehouse name
DATABASE	SALES_DB	Database name
SCHEMA	PUBLIC	Schema to use
ROLE	ACCOUNTADMIN	Role used for the session

Step 3: Install Dependencies in databricks notebook

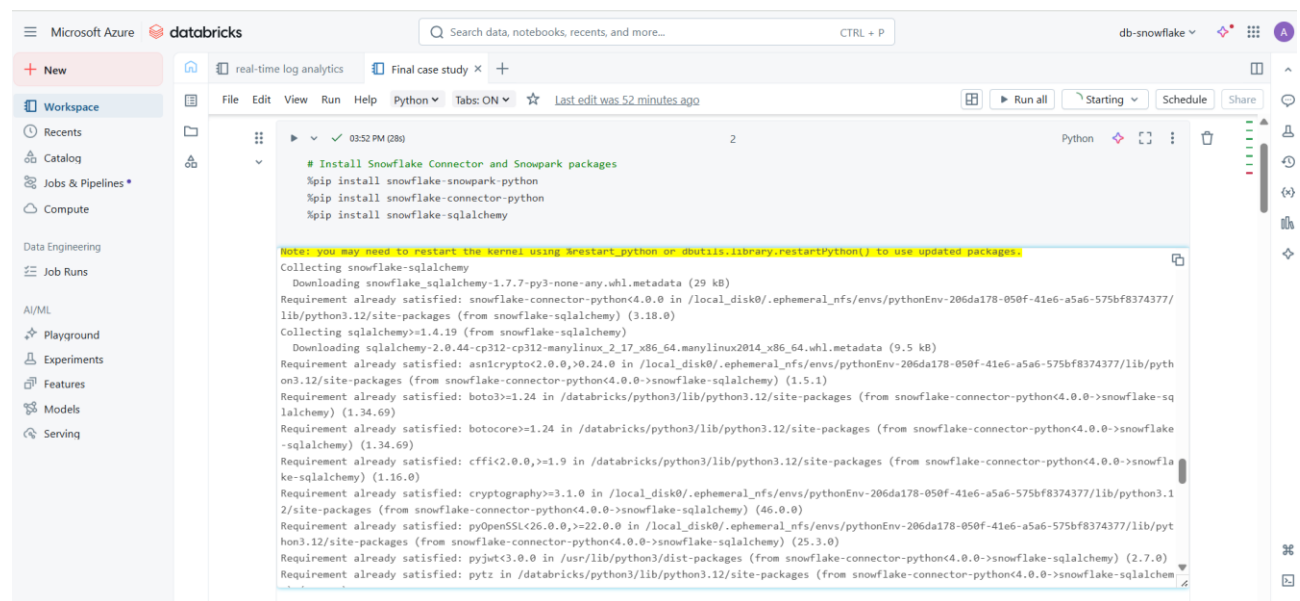
In Databricks, run:

```
%pip install snowflake-snowpark-python
```

```
%pip install snowflake-connector-python
```

```
%pip install snowflake-sqlalchemy
```

This installs Snowpark and its pandas integration.



The screenshot shows a Databricks notebook interface. The top bar includes the Microsoft Azure logo, the Databricks logo, a search bar, and the text 'CTRL + P'. Below the top bar, there's a sidebar with navigation options like 'New', 'Workspace', 'Recents', 'Catalog', 'Jobs & Pipelines', 'Compute', 'Data Engineering', 'Job Runs', 'AI/ML', 'Playground', 'Experiments', 'Features', 'Models', and 'Serving'. The main area displays a notebook with a code cell. The code cell contains the following text:

```
# Install Snowflake Connector and Snowpark packages
%pip install snowflake-snowpark-python
%pip install snowflake-connector-python
%pip install snowflake-sqlalchemy
```

Below the code, there's a yellow warning box that says: "Note: you may need to restart the kernel using %restart_python or dbutils.library.restartPython() to use updated packages." Below the warning box, there's a list of package requirements and their versions, including snowflake-sqlalchemy, sqlalchemy, cryptography, pyOpenSSL, boto3, and pyjwt.

Step 4: Create Snowflake Connection Options in Databricks

Import and Create Session

```
from snowflake.snowpark import Session

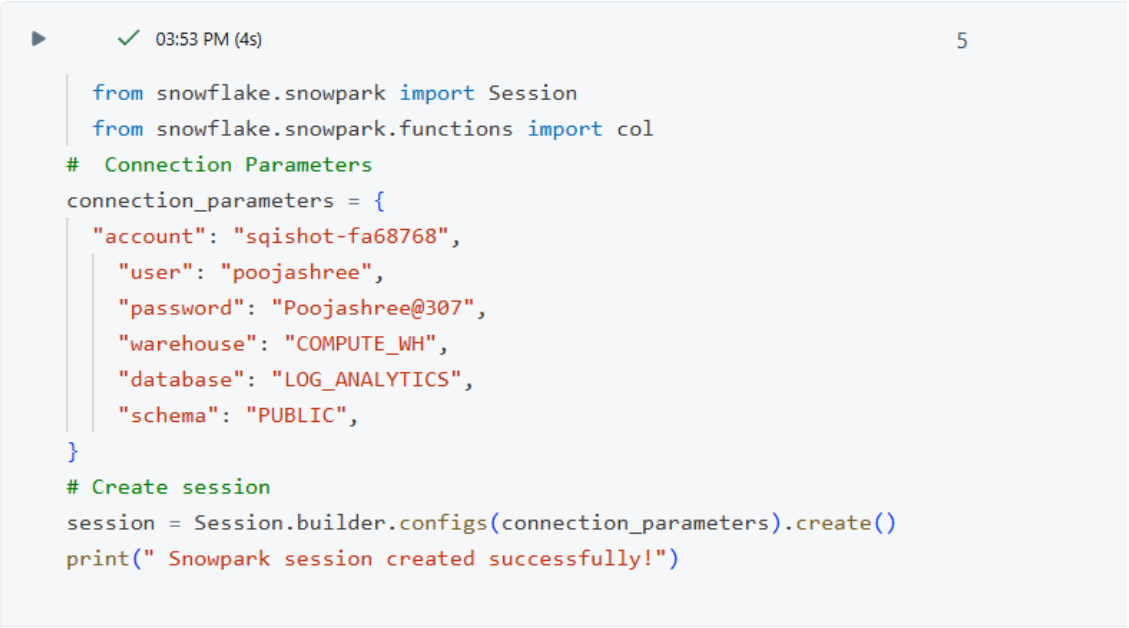
from snowflake.snowpark.functions import col

# Connection Parameters
connection_parameters = {
    "account": "sqishot-fa68768",
    "user": "poojashree",
    "password": "Poojashree@307",
    "warehouse": "COMPUTE_WH",
    "database": "LOG_ANALYTICS",
    "schema": "PUBLIC",
}

# Create session
session = Session.builder.configs(connection_parameters).create()

print(" Snowpark session created successfully!")
```

Import and Create Session



```
from snowflake.snowpark import Session
from snowflake.snowpark.functions import col
# Connection Parameters
connection_parameters = {
    "account": "sqishot-fa68768",
    "user": "poojashree",
    "password": "Poojashree@307",
    "warehouse": "COMPUTE_WH",
    "database": "LOG_ANALYTICS",
    "schema": "PUBLIC",
}
# Create session
session = Session.builder.configs(connection_parameters).create()
print(" Snowpark session created successfully!")
```

Snowpark session created successfully!

Step 5: Read the File from Azure Stage

azure://sfhexastorage.blob.core.windows.net/monthly-sales/

and it contains:

Retail_Sales__500_rows__Preview.csv

Read CSV from stage

```
df_stage = session.read.options({  
    "field_delimiter": ",",  
    "skip_header": 1,  
    "FIELD_OPTIONALLY_ENCLOSED_BY": ""  
}).csv("@RETAIL_DB.SALES_STAGE.azure_stage/Retail_Sales__500_rows__Preview.csv")
```

Show sample data

```
df_stage.show(5)
```

Read the File from Azure Stage

```
df_stage = session.read.options({  
    "field_delimiter": ",",  
    "skip_header": 1,  
    "FIELD_OPTIONALLY_ENCLOSED_BY": ""  
}).csv("@RETAIL_DB.SALES_STAGE.azure_stage/Retail_Sales__500_rows__Preview.csv")  
# Show sample data  
df_stage.show(5)
```

Option 'inferSchema' is aliased to 'INFER_SCHEMA'. You may see unexpected behavior. Please refer to format specific options for more information

"c1"	"c2"	"c3"	"c4"	"c5"	"c6"	"c7"	"c8"	"c9"	"c10"	"c11"	"c12"	"c13"
ORD-5F8D6F0C 780.43	2024-10-08	2024-10	CUST1000	Ananya Sharma	India	South	Mumbai	Office Supplies	Paper	9	0.00	2700.0
ORD-BF0078E4 4135.60	2024-08-11	2024-08	CUST1001	Aarav Iyer	India	Central	Lucknow	Technology	Networking	4	0.15	27200.0
ORD-86CDS8A3 5676.96	2024-06-12	2024-06	CUST1002	Arjun Sharma	USA	East	Kolkata	Furniture	Tables	4	0.10	31500.0
ORD-FB0CD2D9 11783.22	2024-12-18	2024-12	CUST1003	Ananya Das	India	North	Kolkata	Office Supplies	Appliances	9	0.00	36000.0

Step 6: Write Data into a Table

Now let's create your table and load data from the DataFrame:

Define your target table name

```
table_name = "RAW_SALES"
```

Write to table (overwrite or append)

```
df_stage.write.mode("overwrite").save_as_table(table_name)
```

```
print(f" Data successfully written into table: {table_name}")
```

Write Data into a Table

```

# Define your target table name
table_name = "RAW_SALES"

# Write to table (overwrite or append)
df_stage.write.mode("overwrite").save_as_table(table_name)

print(f" Data successfully written into table: {table_name}")

```

Data successfully written into table: RAW_SALES

Step 7: Validate Ingestion and then transforming the data

You can check row counts or view a few rows directly:

```
df = session.table("RAW_SALES")
print(f"Total rows loaded: {df.count()}")
df.limit(5).show()
```

Validate Ingestion

03:56 PM (<1s)

11

```
df = session.table("RAW_SALES")
print(f"Total rows loaded: {df.count()}")

df.limit(5).show()
```

Total rows loaded: 25

"c1"	"c2"	"c3"	"c4"	"c5"	"c6"	"c7"	"c8"	"c9"	"c10"	"c11"	"c12"	"c13"	"c14"
ORD-5F8D6F0C	2024-10-08	2024-10	CUST1000	Ananya Sharma	India	South	Mumbai	Office Supplies	Paper	9	0.00	2700.0	780.43
ORD-BF0078E4	2024-08-11	2024-08	CUST1001	Aarav Iyer	India	Central	Lucknow	Technology	Networking	4	0.15	27200.0	4135.60
ORD-86CD58A3	2024-06-12	2024-06	CUST1002	Arjun Sharma	USA	East	Kolkata	Furniture	Tables	4	0.10	31500.0	5676.96
ORD-FB0CD2D9	2024-12-18	2024-12	CUST1003	Ananya Das	India	North	Kolkata	Office Supplies	Appliances	9	0.00	36000.0	11783.22
ORD-EF35596B	2024-10-27	2024-10	CUST1004	Ishaan Bhat	UK	Central	Chennai	Furniture	Storage	4	0.00	24000.0	4189.98

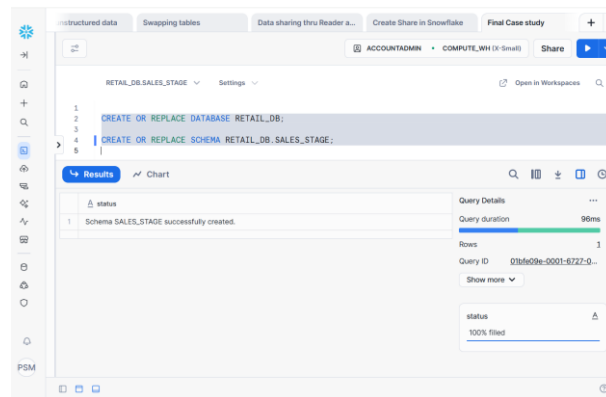
Transform & Model via Snowpark

[illegible]

Step 8: Create Snowflake Objects

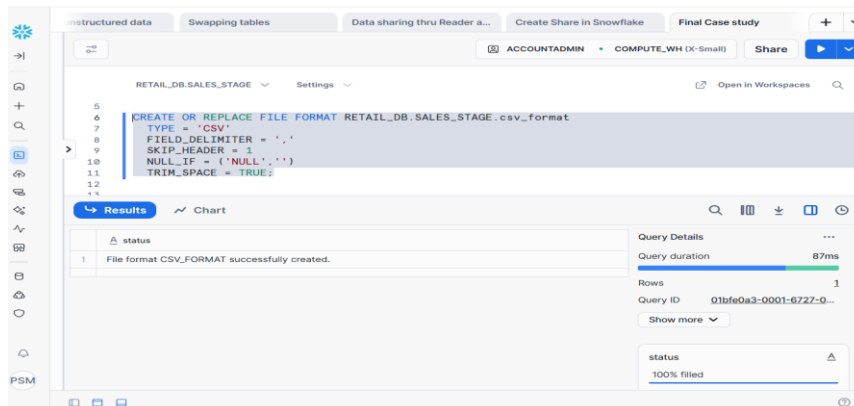
8.1 Database & Schema

```
CREATE OR REPLACE DATABASE RETAIL_DB;  
CREATE OR REPLACE SCHEMA RETAIL_DB.SALES_STAGE;
```



8.2 File Format

```
CREATE OR REPLACE FILE FORMAT RETAIL_DB.SALES_STAGE.csv_format  
TYPE = 'CSV'  
FIELD_DELIMITER = ','  
SKIP_HEADER = 1  
NULL_IF = ('NULL', '')  
TRIM_SPACE = TRUE;
```



8.3 Stage (with SAS URL)

```
CREATE OR REPLACE STAGE RETAIL_DB.SALES_STAGE.azure_stage  
  
URL='azure://sfhexastorage.blob.core.windows.net/monthly-sales'  
  
FILE_FORMAT = RETAIL_DB.SALES_STAGE.csv_format  
  
CREDENTIALS=(AZURE_SAS_TOKEN='?sv=2024-11-  
04&ss=bfqt&srt=sco&sp=rwdlacupyx&se=2025-10-23T17:42:04Z&st=2025-  
10-  
22T09:27:04Z&spr=https&sig=VGd6J9B4ffvNkdB8%2FgkH8mpLQ2Q9ac6  
5xyg6RNNp4DI%3D');
```

The screenshot shows a Snowflake SQL editor interface. The query editor displays the SQL code for creating a stage. Below the query, the 'Results' tab is active, showing a single row with the status 'Stage area AZURE_STAGE successfully created.' The 'Query Details' panel on the right indicates a query duration of 174ms and 1 row returned. The query ID is 01bfe0a8-0001-66d1-0...

status
1 Stage area AZURE_STAGE successfully created.

Query Details

- Query duration: 174ms
- Rows: 1
- Query ID: 01bfe0a8-0001-66d1-0...

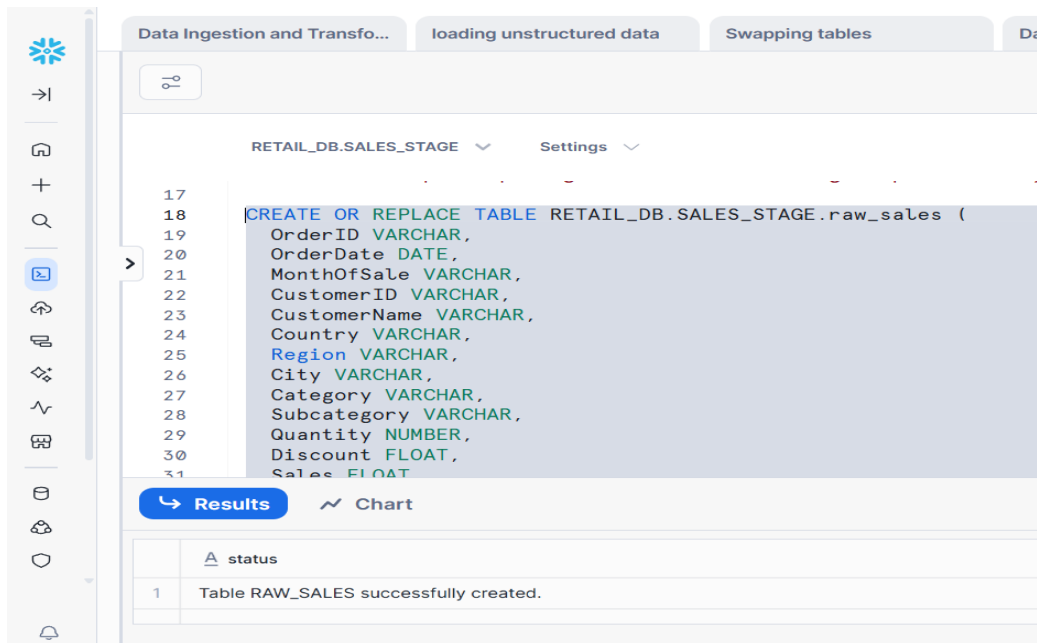
status

100% filled

Step 9: Create Raw Staging Table

```
CREATE OR REPLACE TABLE RETAIL_DB.SALES_STAGE.raw_sales (  
  
OrderID VARCHAR,  
  
OrderDate DATE,  
  
MonthOfSale VARCHAR,  
  
CustomerID VARCHAR,  
  
CustomerName VARCHAR,  
  
Country VARCHAR,  
  
Region VARCHAR,  
  
City VARCHAR,
```

Category VARCHAR,
Subcategory VARCHAR,
Quantity NUMBER,
Discount FLOAT,
Sales FLOAT,
Profit FLOAT
);



Step 10: Connect Power BI to Snowflake

1. Open Power BI Desktop.
2. Click Get Data → Snowflake.
3. Fill in connection details:
 - Server: sqishot-fa68768.snowflakecomputing.com
 - Warehouse: e.g., COMPUTE_WH
 - Database: RETAIL_DB (or LOG_ANALYTICS if using that)
 - Schema: ANALYTICS
4. Click DirectQuery
5. Authenticate using your username/password

Step 11: Load Tables

1. Select the (SALES) tables (CUSTOMER, PRODUCT).
2. Click Load.
3. Power BI will create a data model.

Step 12: Create Measures & Calculated Columns

For business users, useful KPIs:

- Total Sales

Total Sales = SUM(SALES[Sales])

- Total Profit

Total Profit = SUM(SALES[Profit])

- Profit Margin %

Profit Margin % = DIVIDE([Total Profit],[Total Sales],0)

- Discount %

Discount % = AVERAGE(FACT_SALES[Discount])*100

Step 13 : Build Visuals:

1. Sales Trend by Month

- Chart: Line chart
- Axis: SALES.MonthOfSale
- Value: Total Sales

2. Top Customers by Sales

- Chart: Bar chart
- Axis: CUSTOMER.CustomerName
- Value: Total Sales
- Sort descending

3. Category-wise Sales & Profit

- Chart: **Clustered column chart**
- Axis: PRODUCT.Category
- Values: Total Sales & Total Profit

4. KPIs Cards

- Total Sales
- Total Profit
- Average Discount
- Profit Margin %

5. Country wise profit

By using donut chart

The below image is my power bi report :

